به نام خدا

گزارش تمرین کامپیوتری
شماره یک
سیگنال ها و سیستم ها

محمد صدرا عباسی - ۸۱۰۱۰۱۴۶۹

فهرست مطالب

پخش اول

1) Scalar Variables

2) Vector Variables

3) Matrix Variables

4) Scalar Equations

5) Matrix Equations

6) Common functions and indexing

7) Plotting multiple lines and colors

پخش دوم

تمرین ۱)

تمرین ۲)

تمرین ۳)

تمرین ۴)

تمرین ۵)

پخش سوم

تمرین ۱)

تمرین ۲)

تمرین ۳)

تمرین ۴)

پخش چهارم

تمرین ۱)

تمرین ۲)

تمرین ۳)

بخش اول

1) Scalar Variables

```
a = 10;
```

در این یک عدد صحیح تعریف شده است که مقدار 10 را در متغیر a ذخیره می کند.

```
b = 2.5e23;
```

برای نمایش عدد علمی از حرف e استفاده می شود که توان 10 را نشان می دهد.

```
c = 2 + 3i;
```

برای تعریف بخش موهومی یک عدد مختلط از حروف i یا j در MATLAB استفاده می شود.

```
d = exp(1i*2*pi/3);
```

برای عدد نپر از تابع نمایی exp() استفاده شده همچنین برای عدد پی و ریشه دوم -1 به ترتیب از متغیر های pi و i در این خط استفاده شده است.

2) Vector Variables

```
aVec = [3.14, 15, 9, 26];
```

در این خط یک بردار سطری به نام aVec ایجاد شده است که عناصر سطر آن، داخل کروشه قرار گرفته و با ',' از هم جدا شده اند.

```
bVec = [2.71; 8; 28; 182];
```

در این خط یک بردار ستونی به نام bVec ایجاد شده است که برای رفتن از هر سطر به سطر بعدی در این ماتریس از ';' در داخل کروشه استفاده شده است.

```
cVec = 5:-0.2:-5;
```

در این خط، برای ایجاد بردارد با گام های مشخص از ':' با ساختار start:step:end استفاده شده است.

```
dVec = logspace(0, 1, 101);
```

در این خط یک بردار با فاصله های لگاریتمی ایجاد شده است. برای اعداد از 10^0 تا 10^1 با توان هایی که با گام 0.01 اضافه می شوند از تابع logspace(a, b, n) استفاده شده است که در آن n برابر با $\frac{1-0}{0.01} + 1 = 101$ عنصر، توان شروع برابر با صفر و توان پایان برابر با یک است.

```
eVec = 'Hello';
```

در متلب برای اینکه کل متن به عنوان یک string در نظر گرفته شود از double quotation (") استفاده می شود و برای اینکه متن به صورت یک بردارد از کارکتر های مجزا در نظر گرفته شود از

single quotation (') استفاده می شود.

3) Matrix Variables

```
aMat = 2 * ones(9, 9);
```

در این خط یک ماتریس مربعی 9 در 9 شامل اعداد 2 ایجاد شده است. برای اینکار ابتدا با تابع ones() یک ماتریس پر از عدد 1 ساخته شده و سپس اسکالر 2 در کل ماتریس ضرب می‌گردد

```
bMat = diag([1,2,3,4,5,4,3,2,1]);
```

در این خط یک ماتریس مربعی با درایه‌های مشخص بر روی قطر اصلی ایجاد شد. با استفاده از دستور diag()، عناصر بردار ورودی دقیقاً بر روی قطر اصلی قرار گرفته و بقیه عناصر ماتریس صفر در نظر گرفته می‌شوند.

```
cMat = reshape(1:100, 10, 10);
```

در این خط ماتریسی ایجاد شده است که اعداد 1 تا 100 به صورت ستونی در آن قرار گرفته اند. تابع reshape() عناصر تولید شده توسط بازه 1:100 را در قالب جدید یک ماتریس 10 در 10 بازآرایی می‌کند.

```
dMat = NaN(3, 4);
```

در این خط یک ماتریس 3 در 4 شامل مقادیر تعریف نشده یا خالی ایجاد شده است. برای اینکار از تابع Nan() با مشخص کردن ابعاد آن، استفاده شده است.

```
eMat = [13,-1,5; -22,10,-87];
```

در این خط یک ماتریس 2 در 3 با وارد کردن دستی درایه های آن ایجاد شده است. برای رفتن به سطر بعدی از ';' استفاده شده است.

```
% using floor and rand
fMat_floor = floor(7*rand(5,3))-3;
fMat_ceil = ceil(7*rand(5,3))-4;
% using randi
fMat_randi = randi([-3,3], 5, 3);
```

در این خطوط ماتریس‌هایی با ابعاد 3 در 5 متشکل از اعداد صحیح تصادفی تولید شدند. در رویکرد اول، با تغییر مقیاس خروجی تابع rand() و استفاده از توابع گرد کردن به پایین floor() و گرد کردن به بالا ceil() محدوده مطلوب ساخته شده است. در رویکرد نهایی نیز به طور مستقیم از دستور randi() برای استخراج اعداد صحیح تصادفی در بازه [3, 3-] استفاده شده.

4) Scalar Equations

```
x = 1/(1 + exp(-(a-15)/6));
```

در این خط معادله x با استفاده از تابع نمایی exp() محاسبه شده است که در نتیجه آن، مقدار 0.302940716034593 در متغیر x ذخیره شده است.

```
% using nthroot
y_1 = (sqrt(a) + nthroot(b, 21))^pi;
% using power
y_2 = (sqrt(a) + b^(1/21))^pi;
```

برای محاسبه متغیر y، از دو روش مختلف برای ریشه‌گیری استفاده شده است. در خط اول از تابع nthroot() برای محاسبه ریشه 21 ام استفاده شده و در خط دوم از توان کسری استفاده شده است. تفاوت اصلی این دو روش در برخورد با اعداد منفی است؛ تابع nthroot() برای پایه‌های منفی ریشه حقیقی تولید می‌کند، در حالی که توان کسری منجر به تولید جواب مختلط می‌شود (هرچند در این مسئله به دلیل مثبت بودن عدد پایه، خروجی هر دو رویکرد کاملاً یکسان است). همچنین برای ریشه دوم ثابت a از دستور sqrt() استفاده شده. در نتیجه اجرای آن، مقدار 6.269603144202714e در هر دو متغیر ذخیره شده است

```
z = log(real((c+d)*(c-d)) * sin(a*pi/3)) / (c * conj(c));
```

در این خط از تابع real() برای استخراج بخش حقیقی عبارت مختلط، از تابع conj() برای محاسبه مزدوج مختلط متغیر c در مخرج کسر و از دستور log() برای گرفتن لگاریتم طبیعی صورت کسر استفاده شده است که در نتیجه اجرای آن، مقدار 0.104633566196183 در متغیر z ذخیره شده است.

5) Matrix Equations

```
xMat = (aVec*bVec)*(aMat^2);
```

به دلیل اینکه aVec یک بردار سطری و bVec یک بردار ستونی است، نتیجه ضرب آنها یک مقدار اسکالر می شود. سپس این مقدار در ماتریس aMat که به توان دو رسیده(یعنی یکبار در خودش ضرب شده) ضرب می شود که در نتیجه آن، ماتریس 9 در 9 با مقادیر تماما 1.840503384000000e در متغیر x ذخیره می شود

```
yMat = bVec * aVec;
```

در این خط بردار ستونی bVec در بردار سطری aVec ضرب شده بر خلاف حالت قبل، حاصل این ضرب خارجی یک ماتریس 4 در 4 است و نه یک مقدار اسکالر.

```
zMat = det(cMat) * (aMat * bMat)';
```

در این خط از تابع det() برای محاسبه دترمینان ماتریس مربعی cMat استفاده شده.همچنین نماد پریم (') برای گرفتن ترانهاده (Transpose) از حاصل‌ضرب دو ماتریس aMat و bMat به کار رفته است.

6) Common functions and indexing

```
cSum = sum(cMat);
```

در این خط جمع مقادیر هر ستون محاسبه شده است. تابع sum() به صورت پیش‌فرض عناصر هر ستون از ماتریس را جمع کرده و یک بردار سطری به عنوان خروجی ارائه می‌دهد.

```
eMean = mean(eMat,2);
```

در این خط برای محاسبه میانگین روی سطر های ماتریس eMat از تابع mean() با پارامتر 2 استفاده شده که در آن 2 نشان دهنده جهت است یعنی میانگیری در راستای سطر ها انجام می شود و نتیجه آن یک ماتریس ستونی خواهد بود.

```
eMat(1, :) = [1,1,1];
```

در این خط تمام عناصر سطر اول ماتریس eMat انتخاب شده و مقادیر آنها با [1,1,1] جایگزین شده است. درواقع ':' در پارامتر دوم به معنای انتخاب تمام ستون هاست.

```
cSub = cMat(2:9, 2:9);
```

در این خط یک زیر ماتریس با محدوده 2 تا 9 برای سطر ها و محدوده 2 تا 9 برای ستون ها از ماتریس cMat جدا شده است.

```
lin = 1:1:20;
lin(2:2:20) = -lin(2:2:20);
```

ابتدا برداری سطری از 1 تا 20 با گام 1 در متغیر lin ذخیره شده است سپس با استفاده از عملگر دو نقطه و تعیین گام 2، تنها اندیس‌های زوج استخراج شده و مقادیر متناظرشان با اضافه شدن علامت منفی، قرینه شدند.

```
r = rand(1,5);
elements_idx = find(r<0.5);
r(elements_idx) = 0;
```

در این خطوط ابتدا یک بردار تصادفی شامل 5 عنصر تولید شده سپس با استفاده از تابع find() موقعیت عناصری که مقدار آن‌ها کوچکتر از 0.5 بوده در یک متغیر استخراج شده و در نهایت تنها مقدار همان عناصر در بردار اصلی برابر صفر قرار گرفته است.

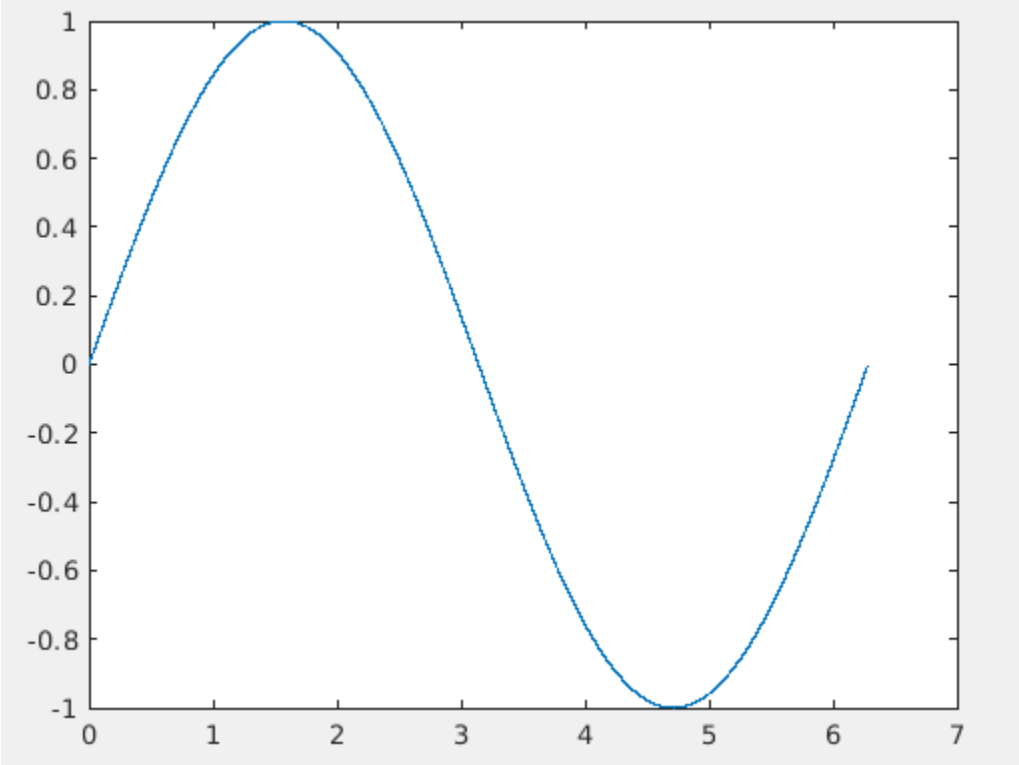
7) Plotting multiple lines and colors

در این قسمت یک اسکریپت برای رسم همزمان دو تابع روی یک محور مختصات نوشته شد. ابتدا با استفاده از دستور figure یک پنجره جدید برای رسم توابع باز شد سپس یک بردار زمان t از صفر تا 2pi با گام 0.001 ایجاد شد تا منحنی ها به شکل نسبتا پیوسته ای رسم شوند:

```
figure;
t = 0:0.001:2*pi;
```

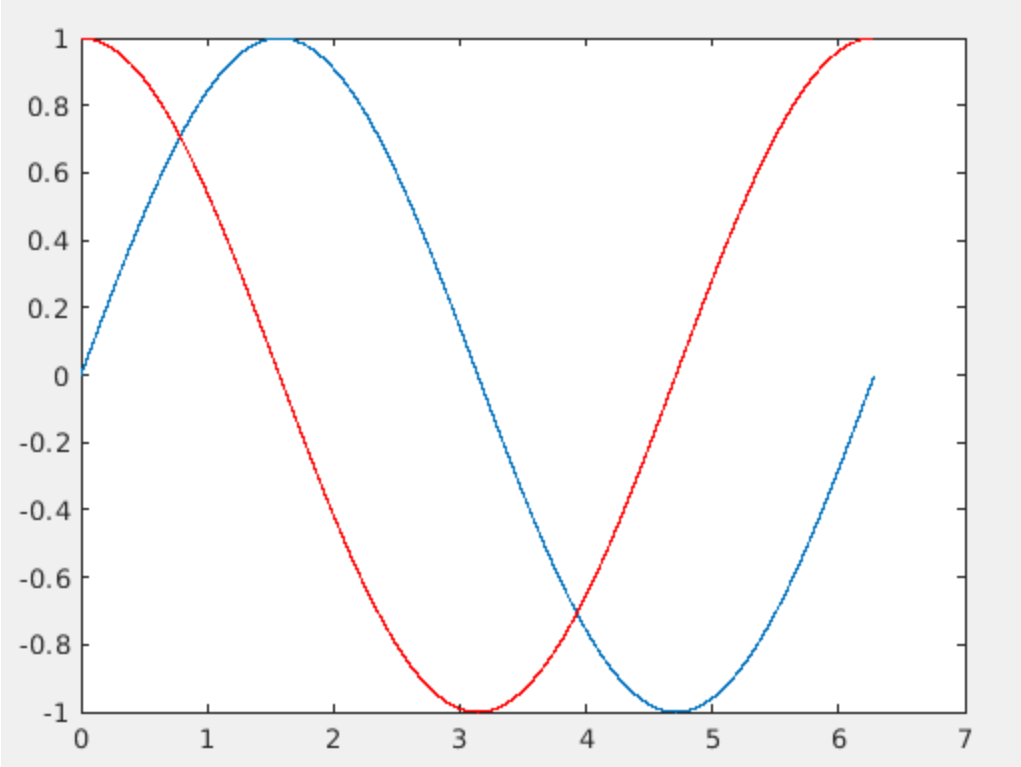
در قسمت بعد مقادیر یک موج سینوس بر حسب t محاسبه شد و در متغیر x ذخیره شد و سپس با دستور plot به صورت پیشفرض به رنگ آبی رسم شد:

```
x = sin(t);
plot(t, x);
```



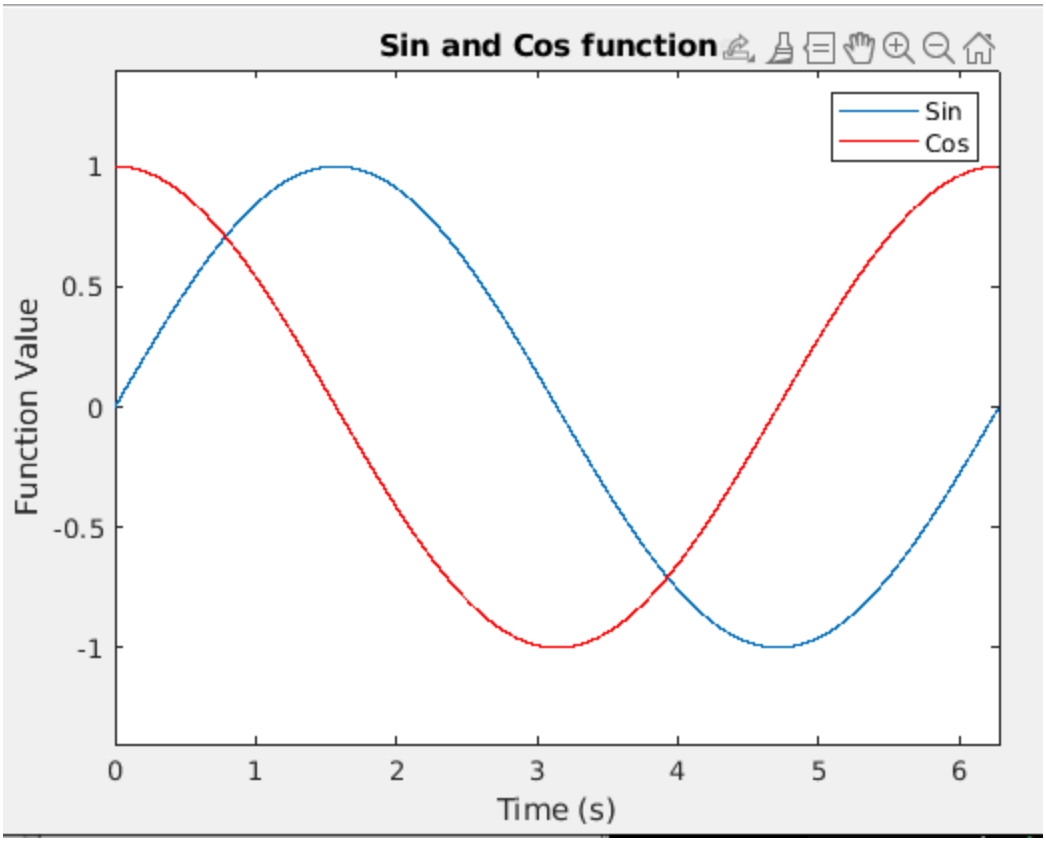
در ادامه با دستور hold on ،شکل نگه داشته شد تا موقع اضافه کردن موج جدید، موج قبلی پاک نشود سپس مقادیر یک موج کسینوس محاسبه شده و با تعیین پارامتر 'r' به رنگ قرمز بر روی مختصات رسم شد:

```
hold on;
y = cos(t);
plot(t, y, 'r');
```



در خطوط زیر برچسب‌گذاری و تنظیمات ظاهری محورها انجام شد. محورهای افقی و عمودی به ترتیب با توابع xlabel و ylabel نام‌گذاری شدند و عنوان نمودار با دستور title در بالای شکل درج گردید. از دستور legend برای ایجاد جعبه راهنما استفاده شد تا مشخص شود هر رنگ مربوط به کدام تابع است. در نهایت، برای داشتن کادربندی دقیق و جلوگیری از ایجاد فضای خالی اضافه، دامنه محور افقی با استفاده از دستور xlim به بازه 0 تا 2 و دامنه محور عمودی با دستور ylim به بازه 1.4- تا 1.4 محدود شد و قابلیت نگه‌داشتن نمودار با hold off به پایان رسید:

```
xlabel('Time (s)');
ylabel('Function Value');
title('Sin and Cos functions');
legend('Sin', 'Cos');
xlim([0, 2*pi]);
ylim([-1.4, 1.4]);
hold off;
```



بخش دوم

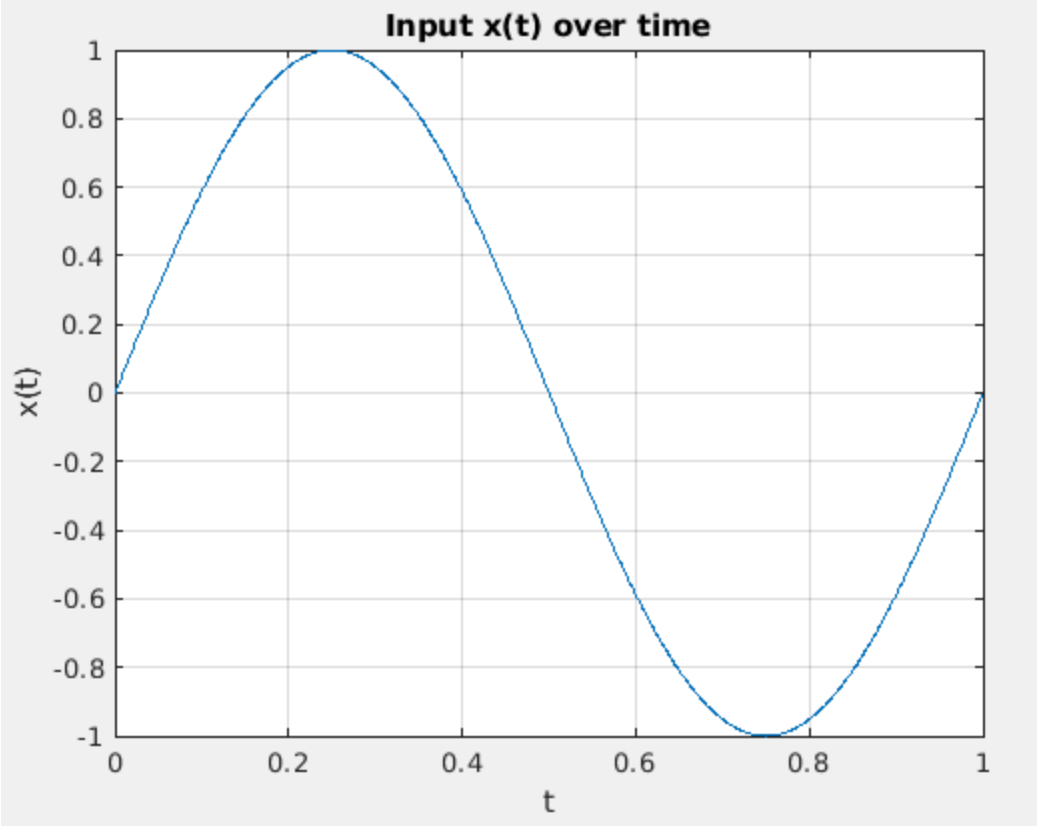
تمرین ۱)

دستور زیر دادهای فایل را در workspace بارگذری می کند:

```
load("p2.mat");
```

سپس در خطوط زیر یک پنجره جدید برای رسم بردار ورودی بر حسب t باز شده و بردار ورودی به شکل یک نمودار پیوسته در آن رسم می شود. دستور grid on حالت شبکه ای صفحه مختصات را برای خوانایی بیشتر فعال می کند:

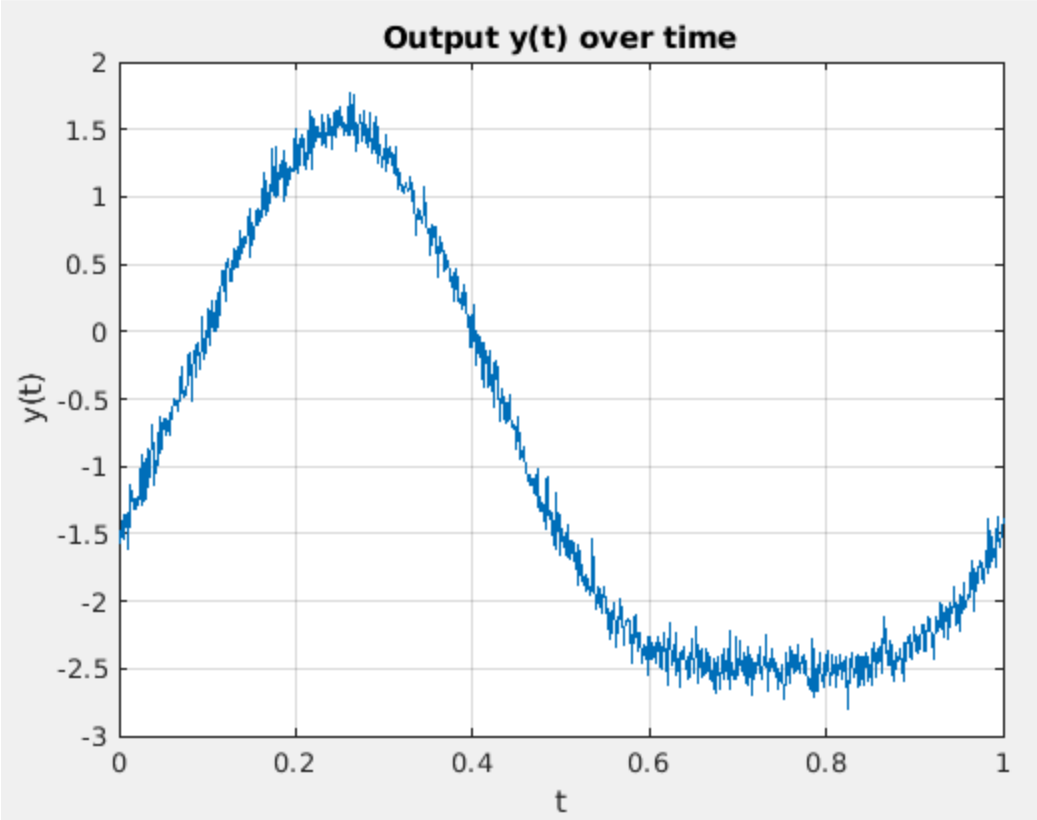
```
figure;
plot(t,x);
xlabel('t');
ylabel('x(t)');
title('Input x(t) over time');
grid on;
```



تمرین ۲)

در این بخش نمودار خروجی بر حسب t رسم شده است. با توجه به نمودار سیگنال خروجی دارای نوسانات تصادفی است که نشان دهنده وجود نویز در سیستم است. خطوط زیر مشابه بخش قبل برای رسم بردار خروجی بر حسب t نوشته شده است:

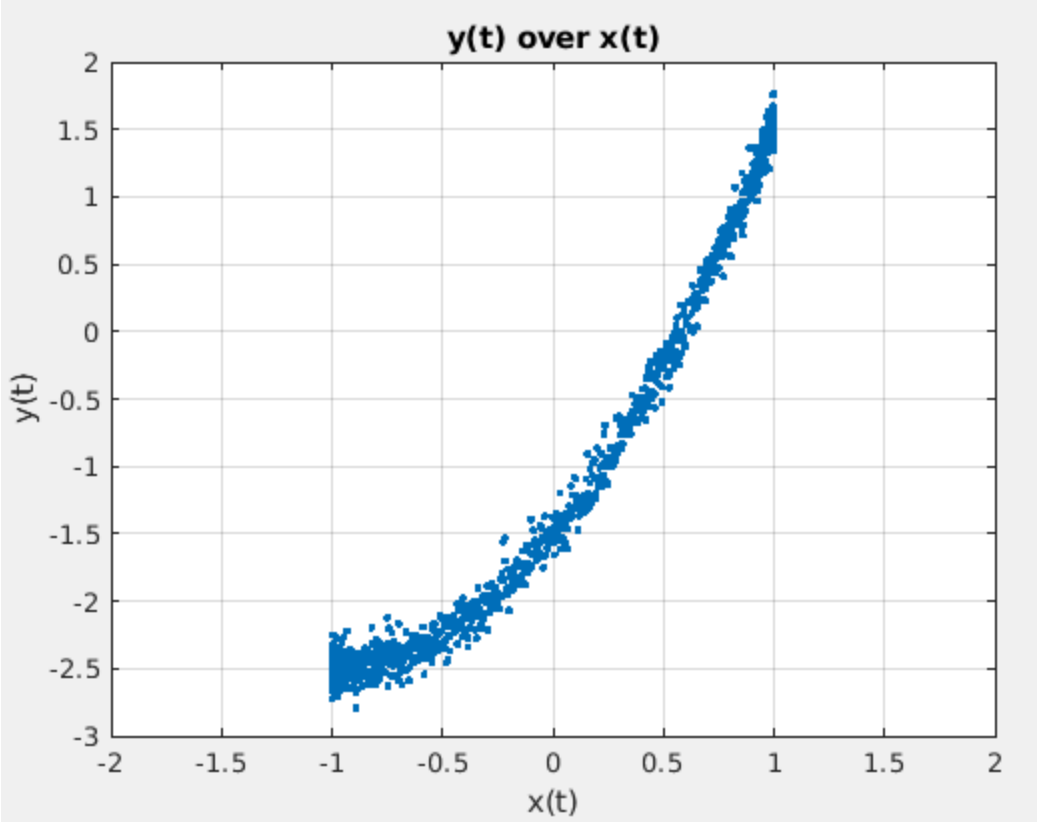
```
figure;
plot(t,y);
xlabel('t');
ylabel('y(t)');
title('Output y(t) over time');
grid on;
```



تمرین ۳)

در این قسمت برای درک بهتر رابطه ورودی و خروجی، نمودار y بر حسب x رسم شده است. در دستور plot استفاده از '.' برای آرگومان سوم باعث می شود تا نقاط به یکدیگر وصل نشوند تا این رابطه را بهتر بتوان مشاهده کرد. با توجه به نمودار و توزیع نقاط می توان تشخیص داد که این رابطه تقریبا به شکل یک معادله درجه ۲است:

```
plot(x, y, '.');
xlabel('x(t)');
ylabel('y(t)');
xlim([-2, 2]);
title('y(t) over x(t)');
grid on;
```



تمرین ۴)

رابطه سیستم در این مسئله به فرم یک معادله درجه دوم تعریف شده است یعنی:

$$y(t) = \alpha(x(t))^2 + \beta\, x(t) + \gamma$$

از آنجا که طبق مشاهدات، داده‌های ما نویزی هستند، نمی‌توان صرفا با جایگذاری سه نقطه در معادله و حل یک دستگاه سه معادله و سه مجهول، پارامترها را استخراج کرد پس برای تخمین پارامترها از روش حداقل مربعات خطا بهره می‌بریم. در واقع در این روش تلاش می‌کنیم ضرایب را به‌گونه‌ای انتخاب کنیم که میزان خطا کمینه شود و بهترین Fit بر روی داده‌ها را بیابیم

ابتدا تابع هزینه را به شکل زیر تعریف می کنیم:

$$f(\alpha, \beta, \gamma) = \sum_t \left(y(t) - \left(\alpha(x(t))^2 + \beta\, x(t) + \gamma\right)\right)^2$$

این تابع هزینه شامل سه بخش اصلی است:

محاسبه انحراف: عبارت داخل پرانتز دقیقا اختلاف میان خروجی واقعی سیستم و خروجی مدل را در هر لحظه محاسبه می‌کند.

توان دوم: برای اینکه خطاهای سمت بالای منحنی (تفاضل‌های مثبت) و خطاهای سمت پایین منحنی (تفاضل‌های منفی) در فرآیند جمع‌زدن یکدیگر را خنثی نکنند، تک‌تک این فواصل به توان 2 می‌رسند.
تجمیع کل: این خطاهای به توان رسیده را برای تمامی نمونه‌های زمانی با هم جمع می شوند

برای محاسبه نقطه مینیمم باید از تابع هزینه نسبت به هر سه پارامتر مشتق جزئی گفته و مساوی صفر قرار دهیم:

۱. مشتق نسبت به γ:

$$\frac{\partial f}{\partial \gamma} = -2\sum_{i=1}^{N}(y_i - \alpha x_i^2 - \beta x_i - \gamma) = 0$$

- تقسیم بر -2 و پخش کردن سیگما(اپراتور خطی):

$$\sum y_i - \alpha\sum x_i^2 - \beta\sum x_i - \sum\gamma = 0$$

- $\sum\gamma$ برای N داده برابر با $N\gamma$:

$$\alpha\sum x_i^2 + \beta\sum x_i + \gamma N = \sum y_i$$

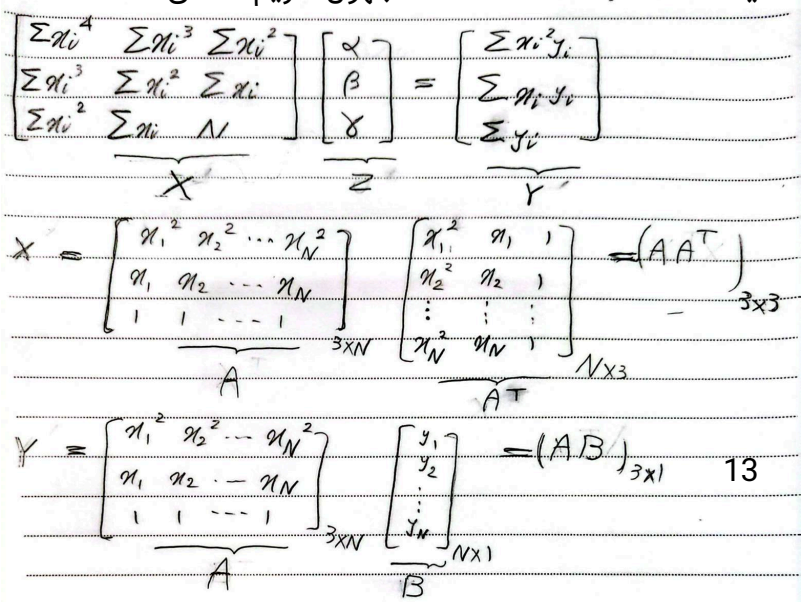
۲. مشتق نسبت به β:

$$\frac{\partial f}{\partial \beta} = -2\sum_{i=1}^{N} x_i(y_i - \alpha x_i^2 - \beta x_i - \gamma) = 0 \Rightarrow \alpha\sum x_i^3 + \beta\sum x_i^2 + \gamma\sum x_i = \sum x_i y_i$$

۳. مشتق نسبت به α:

$$\frac{\partial f}{\partial \alpha} = -2\sum_{i=1}^{N} x_i^2(y_i - \alpha x_i^2 - \beta x_i - \gamma) = 0 \Rightarrow \alpha\sum x_i^4 + \beta\sum x_i^3 + \gamma\sum x_i^2 = \sum x_i^2 y_i$$

حال با توجه به اینکه مقادیر $\sum$ ها مشخص است یک دستگاه سه معادله سه مجهول داریم که می توانیم آن را **به فرم ماتریسی بازنویسی کنیم**:



در نتیجه، برای استخراج بردار مجهولات، کافیست وارون ماتریس مربعی X را از سمت چپ در بردار سمت راست معادله ضرب کنیم: $X_{Mat}Z = Y_{Mat} \Rightarrow Z = X_{Mat}^{-1}Y_{Mat}$

در ادامه به پیاده سازی این منطق در تابع می پردازیم:

```
function [alpha, beta, gama] = p2_4(x, y)
  aMat = [x.^2; x; ones(size(x))];
```

ماتریس اولیه ایجاد می‌شود. این ماتریس از سه سطر تشکیل شده است؛ سطر اول توان دوم درایه‌های بردار ورودی (استفاده از عملگر .^ برای محاسبه درایه به درایه)، سطر دوم خود بردار ورودی و سطر سوم شامل برداری از اعداد یک (با طولی برابر با طول بردار ورودی) است.

```
  xMat = aMat * aMat';
```

ماتریس در ترانهاده خودش (علامت ' در متلب) ضرب ماتریسی می‌شود تا ماتریس مربعی 3 در 3 ضرایب (سمت چپ دستگاه معادلات) شکل گیرد.

```
  yMat = aMat * y';
```

ماتریس در ترانهاده بردار خروجی ضرب می‌شود تا بردار ستونی سمت راست دستگاه معادلات تولید گردد.

```
  zMat = inv(xMat) * yMat;
```

دستگاه معادلات خطی حل می‌شود. با استفاده از تابع inv()، معکوس ماتریس مربعی ضرایب محاسبه شده و از سمت چپ در بردار سمت راست معادله ضرب می‌گردد تا بردار مجهولات به دست آید.

```
  alpha = zMat(1);
  beta = zMat(2);
  gama = zMat(3);
end
```

در این سه خط، پارامترهای تخمین‌زده‌شده به تفکیک از بردار جواب استخراج شده و به عنوان خروجی‌های نهایی تابع برگردانده می‌شوند.

در ادامه به تست ضرایب تخمین زده شده توسط تابع می پردازیم:

برای بررسی دقت عملکرد تابع، از داده های آزمایشی استفاده کرده و نتیجه را با مقداری که به صورت آزمایشی در نظر گرفتیم مقایسه می کنیم.

```
xVec = 0:0.01:10;
alpha = 2.5;
beta = -3;
gama = 1.5;
```

برای بردار ورودی و ضرایب آلفا، بتا و گاما مقادیر دلخواه قرار می دهیم

```
noise = 0.1 * randn(size(xVec));
yVec = alpha*(xVec.^2) + beta*xVec + gama + noise;
```

با استفاده از تابع randn یک نویز تولید تصادفی تولید می کنیم و با ضریب 0.01 آن را در دامنه ای پایین تنظیم می کنیم

```
[alphaEst , betaEst , gamaEst] = p2_4(xVec,yVec);
disp(["estimated alpha = " + alphaEst ; "estimated beta = " + betaEst ; "estimated gama = " + gamaEst]);
```

بردار ورودی و خروجی تولید شده را به عنوان ورودی به تابعی که در بخش قبلی آن را پیاده سازی کردیم می دهیم تا مقادیری که برای **ضرایب آلفا، بتا و گاما تخمین زده را مقایسه و ارزیابی کنیم**:

```
Command Window
    "estimated alpha = 2.5003"
    "estimated beta = -3.0009"
    "estimated gama = 1.4949"

fx >>
```

با توجه به اینکه مقادیر تخمین زده شده توسط تابع به مقادیر آزمایشی ای که تعیین کرده بودیم بسیار نزدیک هستند می توان گفت که عملکرد تابع قابل قبول است.

تمرین ۵)

ابتدا بردار ورودی و خروجی ای که در فایل p2.mat قرار دارد را به عنوان ورودی داده و مقادیر تخمین زده شده برای ضرایب چند جمله را چاپ می کنیم:

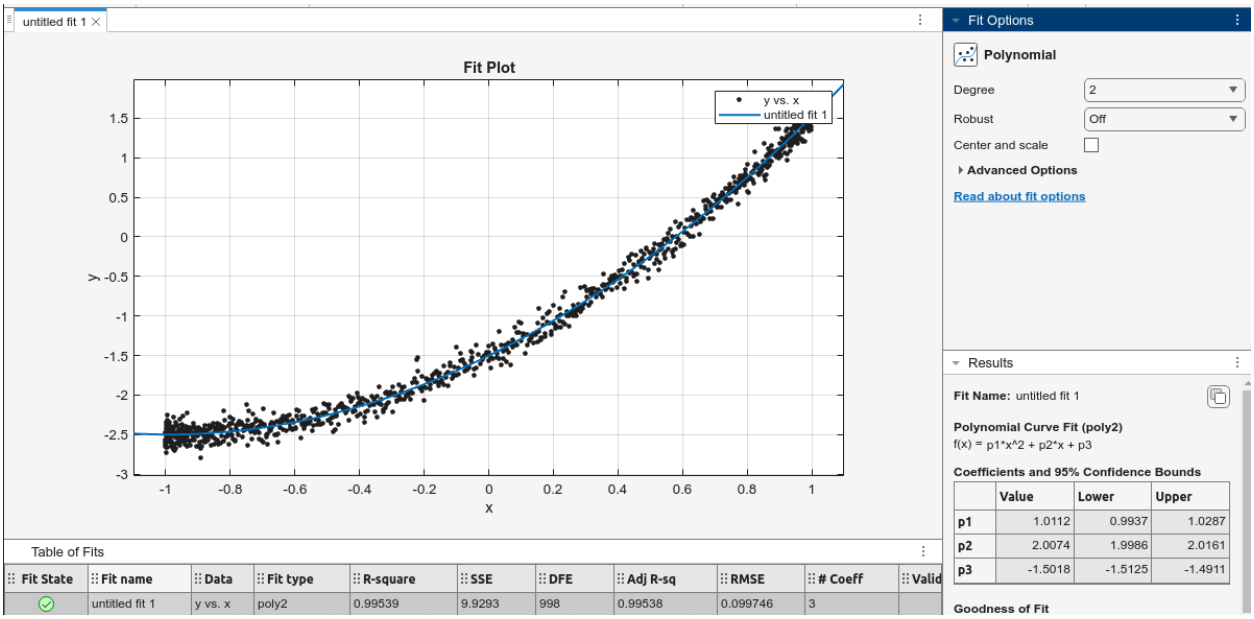
```
[alphaEst , betaEst , gamaEst] = p2_4(x,y);
disp(["estimated alpha = " + alphaEst ; "estimated beta = " + betaEst ; "estimated gama = " + gamaEst]);
```

خروجی این قسمت به شرح زیر است:

```
Command Window
    "estimated alpha = 1.0112"
    "estimated beta = 2.0074"
    "estimated gama = -1.5018"

fx >>
```

حال با استفاده از ابزار Curve Fitting در این بخش ضرایب را بدست می آوریم:



Fit Name: untitled fit 1

Polynomial Curve Fit (poly2)

f(x) = p1*x^2 + p2*x + p3

Coefficients and 95% Confidence Bounds

	Value	Lower	Upper
p1	1.0112	0.9937	1.0287
p2	2.0074	1.9986	2.0161
p3	-1.5018	-1.5125	-1.4911

Goodness of Fit

	Value
SSE	9.9293
R-square	0.9954
DFE	998.0000
Adj R-sq	0.9954
RMSE	0.0997

با در کنار هم گذاشتن این دو نتیجه مشاهده می کنیم که مقادیر تخمین زده شده توسط تابعی که در بخش قبل نوشته شد، مقادیری که ابزار Curve Fitting برای ضرایب بدست آورده دقیقا برابر است و این نشان می دهد که نتیجه گیری ای که درباره عملکرد تابع در تمرین ۴ انجام دادیم کاملا درست است

	Value
p1	1.0112
p2	2.0074
p3	-1.5018

```
Command Window
    "estimated alpha = 1.0112"
    "estimated beta = 2.0074"
    "estimated gama = -1.5018"

fx >>
```

بخش سوم

تمرین ۱)

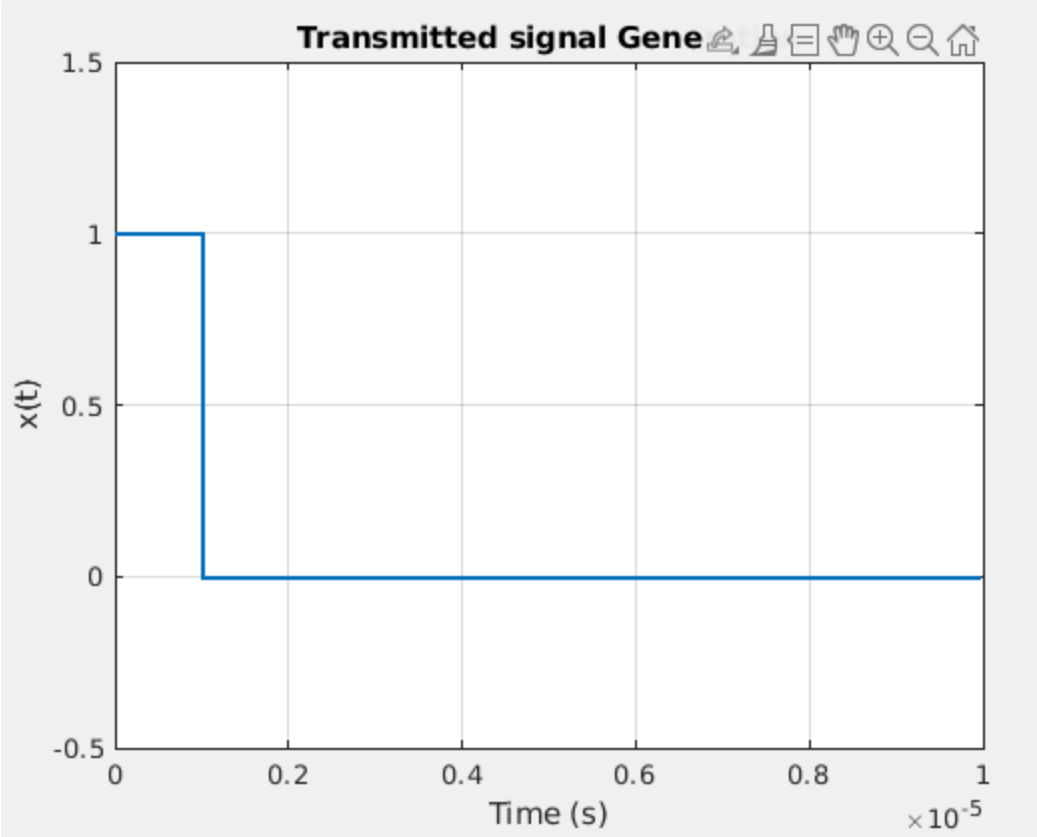
در این ابتدا پارامتر های لازم شامل زمان نمونه برداری(ts)، زمان کل شبیه سازی(T) و عرض پالس(tau) را مقدار دهی می کنیم:

```
clc;
clear;
ts = 1e-9;
T = 1e-5;
tau = 1e-6;
t_len = round(T/ts);
tau_len = round(tau/ts);
t = (0 : t_len-1) * ts;
```

برای پیاده سازی بردار ارسالی لازم است بردار زمان(t) و طول آرایه ها شامل زمان کل(t_len) و عرض پالس (tau_len) را محاسبه کنیم همچنین از تابع round برای تضمین کردن مقدار صحیح این متغیر ها و جلوگیری از warning استفاده شده است.

در ادامه بردار x را ابتدا با صفر مقدار دهی کرده و پالس را با مقدار دهی یک برش از این بردار از 1 تا tau_len ایجاد می کنیم:

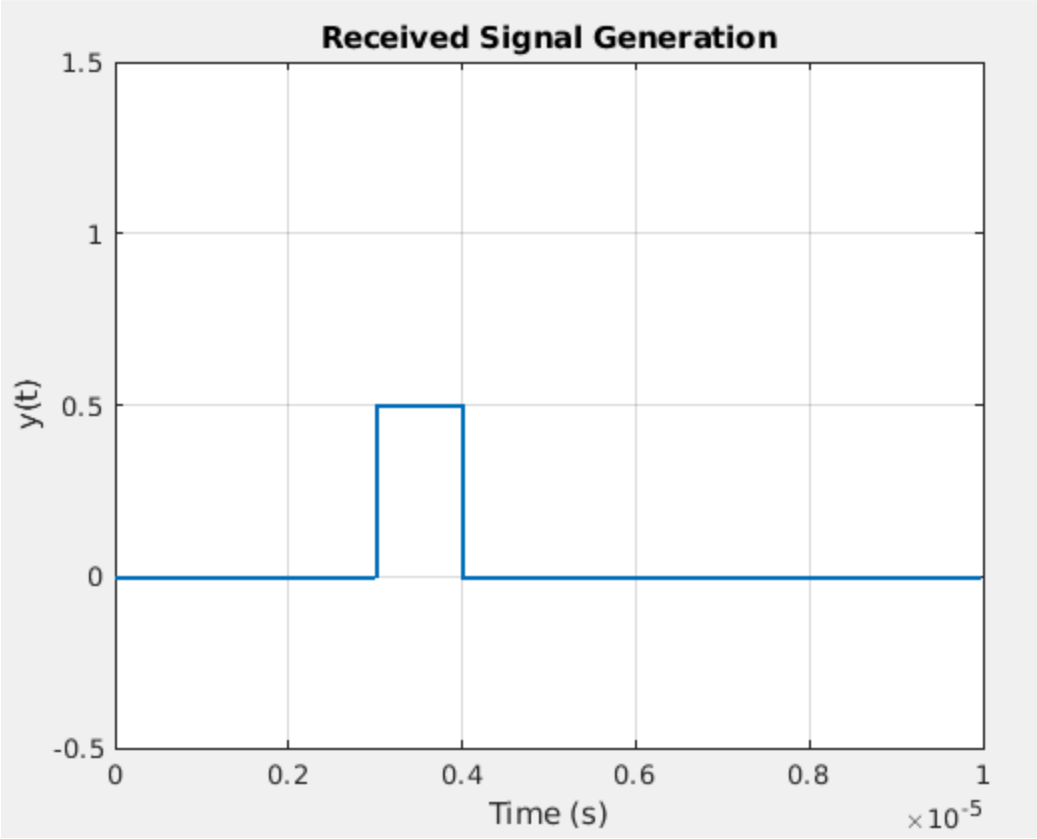
```
x = zeros(1, t_len);
x(1:tau_len) = 1;
```



تمرین ۲)

برای پیاده‌سازی این قسمت، فاصله تا هدف $R = 450$ متر، سرعت نور $C = 3 \times 10^8$ متر بر ثانیه و ضریب دامنه $\alpha = 0.5$ در نظر گرفته شد. با استفاده از رابطه $t_d = \frac{2R}{C}$، زمان رفت و برگشت موج محاسبه شده و سیگنال دریافتی به شکل زیر خواهد بود:

```
R = 450;
C = 3e8;
alpha = 0.5;
td = 2 * R / C;
td_index = round(td / ts) + 1;
y = zeros(1, t_len);
y(td_index : td_index + tau_len - 1) = alpha;
```



تمرین ۳)

در این قسمت، فاصله هدف را مجهول در نظر گرفته و با در دست داشتن سیگنال دریافتی، فاصله را تخمین می‌زنیم. برای این کار از مفهوم استفاده می‌شود. در این روش، سیگنال ارسالی به عنوان الگو (Template) در نظر گرفته می‌شود. این الگو در طول زمان بر روی سیگنال دریافتی لغزانده شده و در هر گام ، میزان شباهت میان الگو و سیگنال دریافتی با استفاده از ضرب داخلی محاسبه می‌شود. نقطه‌ای که به ازای آن مقدار ضرب داخلی بیشینه شود، لحظه‌ای است که الگوی ارسالی با سیگنال بازگشتی بیشترین هم‌پوشانی را دارد؛ بنابراین می‌توان این لحظه را به عنوان زمان تاخیر رفت و برگشت سیگنال (td) در نظر گرفت و در ادامه با استفاده از رابطه $R = \frac{c \cdot t_d}{2}$، فاصله تا هدف را تخمین زد.

دو فرض ساده‌ساز در این پیاده‌سازی در نظر گرفته شده است:

۱. حضور قطعی هدف و سیگنال برگشتی بدون نویز: فرض شده است که حتما یک هدف وجود داشته و سیگنال دریافتی فاقد هرگونه نویز است؛ در غیر این صورت الگوریتم ممکن است بیشینه نویزهای تصادفی را به اشتباه به عنوان زمان تاخیر (td) در نظر بگیرد.

۲. عدم وجود موانع جانبی: فرض شده است که هیچ هدف دیگری در مسیر سیگنال وجود نداشته باشد.

```
template = x(1:tau_len);
```

بخش غیر صفر سیگنال ارسالی به عنوان الگو در نظر گرفته شده است.

```
corr_len = t_len - tau_len + 1;
ro = zeros(1, corr_len);
for i=1:t_len-tau_len
  ro(i) = innerProduct(y(i:i+tau_len-1),template);
end
```

این حلقه به اندازه طول سیگنال منهای طول پالس تکرار می شود تا الگو از ابتدا تا انتهای سیگنال دریافتی حرکت کند. در هر تکرار از حلقه یک قسمت هم اندازه با الگو از سیگنال دریافتی جدا شده و با استفاده از تابع innerProduct ضرب داخلی آن با الگو محاسبه شده و و یک مقدار اسکالر به عنوان میزان شباهت در ایندکس i ام آرایه ro ذخیره می شود

```
[value , index] = max(ro);
```

بعد از اتمام حلقه، ro شامل مقادیر correlation در تمام لحظات است. مقدار و ایندکس بیشترین عنصر در متغیر های value و index ذخیره می شود

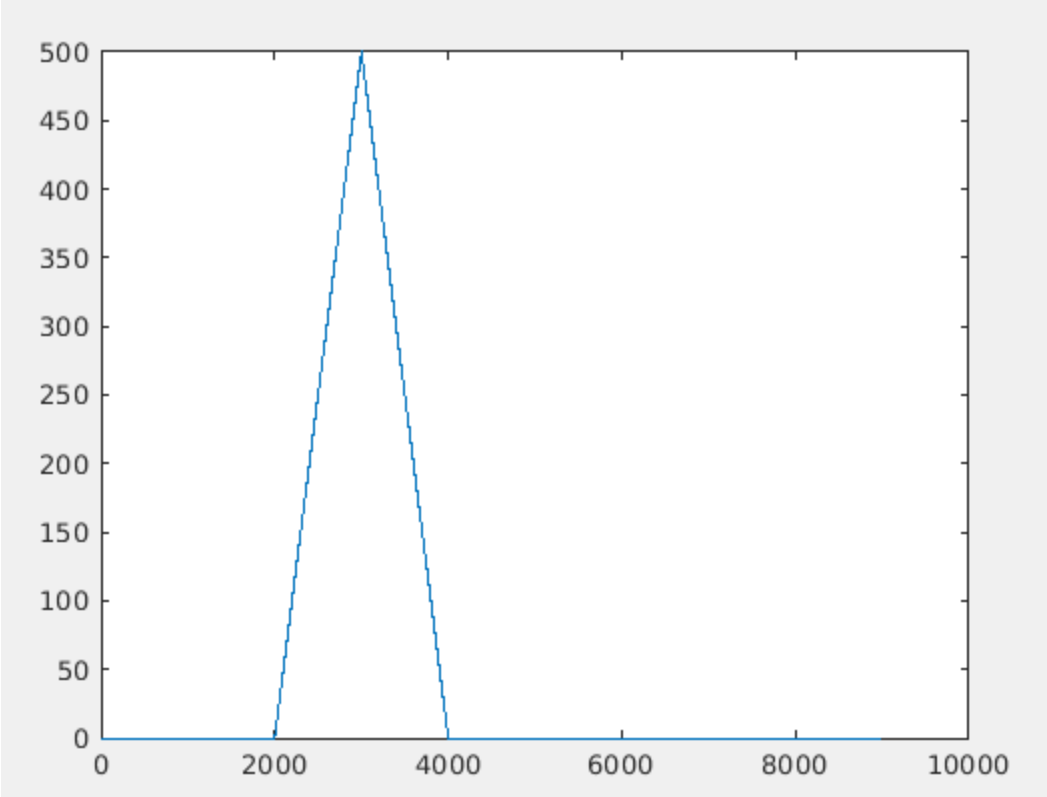
```
calc_td = (index - 1) * ts;
calc_R = (calc_td * C) / 2;
```

با توجه به اینکه اندیس ها از 1 شروع می شوند، با کم کردن مقدار 1 از index و ضرب آن در زمان نمونه برداری (ts)، زمان تاخیر (td) محاسبه شده و سپس با قرار دادن در رابطه $R = \frac{c \cdot t_d}{2}$ مقدار R محاسبه می شود

```
disp(["Calculated td = " + calc_td; "Calculated Distance R = " + calc_R]);
plot(ro);
function ro=innerProduct(a,b)
  ro= sum(a .* b);
end
```

خروجی این قسمت به شکل زیر است:

```
"Calculated td = 3e-06"
"Calculated Distance R = 450"
```



با توجه به اینکه فاصله محاسبه شده 450 متر برابر با فاصله است که در بخش قبل برای تولید سیگنال برگشتی استفاده شده بود، می توان نتیجه گرفت این روش در شرایط ایده آل دقت ۱۰۰ درصد داشته است.

تمرین ۴)

برای این بخش یک نویز گاوسی با میانگین صفر و انحراف معیار σ با استفاده از تابع randn تولید و مدل سازی شده است:

$$z(t) = y(t) + \sigma \cdot w(t), w(t) \sim N(0,1)$$

برای اینکه این نویز به صورت کاملا تصادفی تولید می شود نمی توان از یک بار شبیه سازی نتیجه گیری دقیق انجام داد در نتیجه به ازای هر مقدار ثابت σ الگوریتم را در یک حلقه 100 تایی اجرا می شود. در هر تکرار، یک نویز تصادفی مستقل تولید شده و به سیگنال دریافتی اضافه می شود.

مقادیر σ در این آزمایش به شکل زیر تعریف شده است:

$$SIGMA = [0,\ 0.001,\ 0.01,\ 0.1,\ 1,\ 2,\ 3,\ 4,\ 5,\ 10]$$

در نهایت، میانگین خطای این ۱۰۰ اجرا به عنوان خطای آن سطح نویز تعیین می‌گردد.

طبق معیار مسئله، هرگاه خطای میانگین کمتر از 10 متر باشد، تخمین رادار قابل قبول تلقی می‌شود.

```
SIGMA = [0 0.001 0.01 0.1 1 2 3 4 5 10];
```

سطوح مختلف نویز را مشخص می‌کند. مقادیر از نویز صفر شروع شده و تا نویز بسیار شدید افزایش می‌یابند.

```
SIGMA_len = length(SIGMA);
results = zeros(1, SIGMA_len);
```

یک بردار خالی برای ذخیره میانگین نهایی خطا به ازای هر سطح نویز.

```
template = x(1 : tau_len);
```

بخش مفید پالس ارسالی را به عنوان الگو جدا می‌کند تا با سیگنال دریافتی مقایسه شود.

```
corr_len = t_len - tau_len + 1;
```

تعداد دفعاتی که می‌توان الگو را روی سیگنال دریافتی به اندازه یک نمونه به جلو حرکت داد بدون اینکه از انتهای سیگنال بیرون بزند.

```
TryItr = 100;
```

تعداد دفعات تکرار آزمایش برای هر سطح نویز

```
for i = 1 : SIGMA_len
  errors = zeros(1, TryItr);
```

یک آرایه خالی 100 تایی برای ذخیره خطای تک‌تک اجراهای این سطح نویز.

```
  for j = 1 : TryItr
    noise = SIGMA(i) * randn(1, t_len);
    z = y + noise;
```

تولید بردار نویز تصادفی با میانگین صفر و دامنه متناسب با SIGMA(i) و اضافه کردن نویز به سیگنال بازگشتی

```
    ro = zeros(1, corr_len);
    for k = 1 : corr_len
      ro(k) = innerProduct(z(k : k+tau_len-1), template);
    end
```

یک برش از سیگنال دریافتی (به اندازه طول پالس) برداشته شده و ضرب داخلی آن با پالس الگو محاسبه می‌شود. هرچه این دو بخش شبیه‌تر باشند، خروجی این ضرب عدد بزرگ‌تری خواهد بود.

```
    [value, index] = max(ro);
    calc_td = (index - 1) * ts;
    calc_R = calc_td * C / 2;
    errors(j) = abs(R - calc_R);
  end
```

با یافتن اندیس بیشینه ، زمان تاخیر رفت و برگشت پالس به صورت td=(index−1)×ts محاسبه می‌شود سپس با رابطه R=2c⋅td فاصله تخمینی محاسبه شده و در نهایت خطای مطلق تخمین از اختلاف آن با فاصله واقعی (450 m) به دست می‌آید.

```
  results(i) = mean(errors);
```

پس از اتمام 100 تکرار، میانگین خطای این سطح نویز محاسبه و ذخیره می‌شود.

```
end
figure;
semilogx(SIGMA, results, '-o', 'LineWidth', 1.5, 'MarkerSize', 6);
```

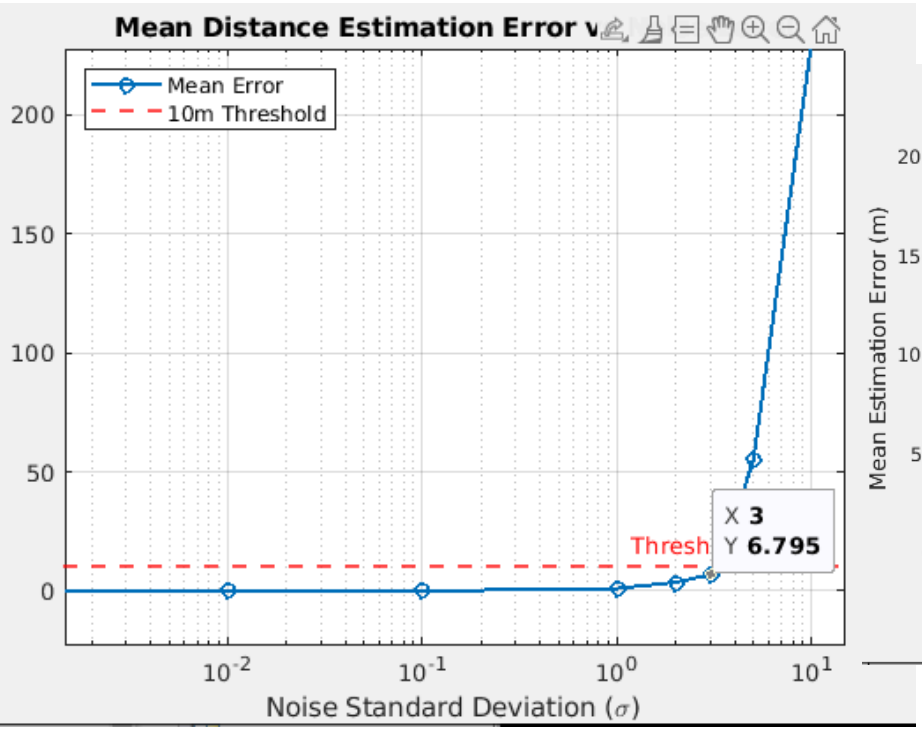
محور افقی را در مقیاس لگاریتمی رسم می‌کند

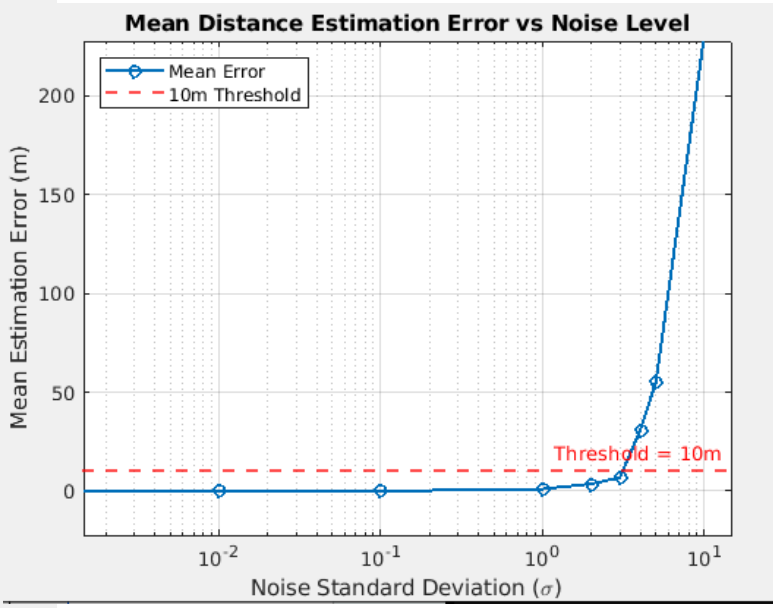
```
hold on;
yline(10, 'r--', 'Threshold = 10m', 'LineWidth', 1.5);
```

یک خط‌چین افقی روی عدد 10 رسم می‌کند تا مرز مجاز مشخص شود و بتوان دید نمودار تا چه نویزی زیر این مرز باقی می‌ماند.

```
xlabel('Noise Standard Deviation (\sigma)');
ylabel('Mean Estimation Error (m)');
title('Mean Distance Estimation Error vs Noise Level');
legend('Mean Error', '10m Threshold', 'Location', 'northwest');
grid on;
```

نتیجه اجرا به صورت زیر خواهد بود:





با توجه به نمودار، در مقادیر پایین نویز (sigma <= 1)، میانگین خطای تخمین تقریبا صفر است در نتیجه الگوریتم می‌تواند فاصله هدف را با دقت مناسبی تشخیص دهد. با افزایش انحراف معیار نویز به sigma=3، خطای تخمین به 6.795 می‌رسد. با وجود افزایش خطا، این مقدار همچنان کمتر از حد مجاز 10 متر است. اما بعد از این بازه، خطا افزایش می‌یابد و این تغییرات نشان‌دهنده افت شدید عملکرد سیستم در صورت نویز زیاد است. دلیل این افزایش خطا این است که با افزایش نویز، احتمال ایجاد پیک های تصادفی بیشتر می‌شود. در نتیجه، الگوریتم ممکن است یک پیک نویزی را انتخاب کند. این اتفاق باعث خطای زیاد در تعیین فاصله می‌شود.

در کل می توان گفت بیشترین مقدار نویز قابل قبول در میان نقاط بررسی‌شده، **sigma=3** است.

بخش چهارم

تمرین ۱)

برای اینکه سیگنال پیوسته صوت بتواند ذخیره سازی و پردازش شود لازم است که با نرخ مشخصی از آن نمونه برداری شده و گسسته در زمان شود. در این بخش هدف پیدا کردن این نرخ نمونه برداری است.

```
[x, fs] = audioread("my voice.wav");
disp("Sampling frequency = " + fs + " Hz");
```

تابع audioread در این خط دو خروجی اصلی دارد. اول بردار داده های سیگنال که دامنه نمونه ها را در طول زمان نگه می دارد و دوم فرکانس نمونه برداری(fs) که نشان دهنده تعداد نمونه های ثبت شده در هر ثانیه از سیگنال پیوسته صوت است.

خروجی این اجرا به شکل زیر است:

```
Command Window
  Sampling frequency = 48000 Hz
fx >>
```

می توان نتیجه گرفت که فاصله زمانی بین دو نمونه متوالی برابر $ts = \frac{1}{fs} = \frac{1}{48000} = 20.83\mu s$ است.

تمرین ۲)

برای اینکه سیگنال را بر حسب زمان رسم کنیم باید نمونه ها به بردار زمان نگاشت شوند از آنجایی که می دانیم فاصله زمانی هر دو نمونه متوالی برابر ts است می توانیم بردار زمان را به صورت زیر تولید کنیم:

$$t = [0\ to\ N - 1] \times ts$$

همچنین با تابع sound(x,fs)، سیگنال صوتی را با فرکانس fs بازپخش می کنیم و با تابع audiowrite سیگنال نهایی x را به شکل یک فایل .wav ذخیر می کنیم

اسکریپت نوشته شده برای این بخش:

```
ts = 1/fs;
samples_num = length(x);
```

محاسبه دوره تناوب نمونه‌برداری و تعداد کل نمونه‌ها

```
duration_sec = (samples_num - 1) * ts;
t = (0 : samples_num - 1) * ts;
```

محاسبه مدت زمان کل سیگنال بر حسب ثانیه و تولید بردار زمان

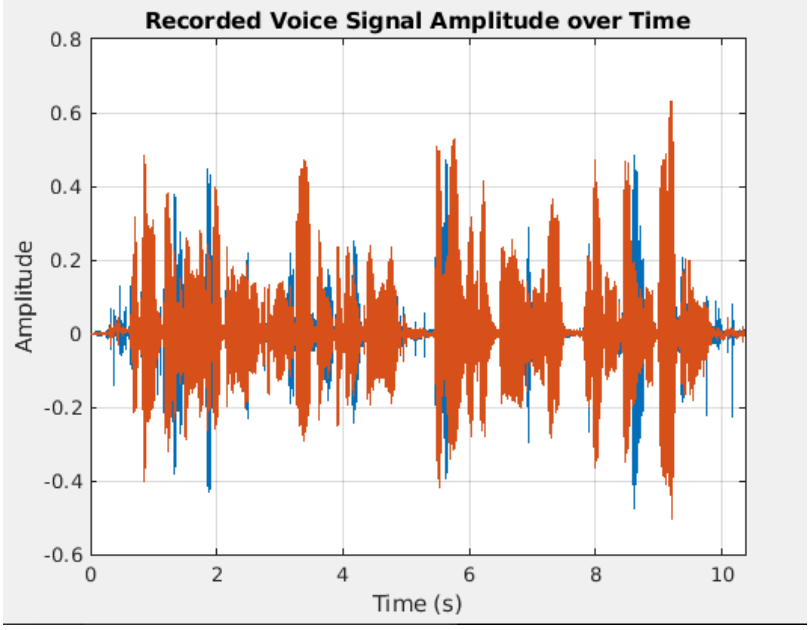
```
figure;
plot(t, x, 'LineWidth', 0.5);
xlabel('Time (s)');
ylabel('Amplitude');
title('Recorded Voice Signal Amplitude over Time');
xlim([0, duration_sec]);
grid on;
```

رسم نمودار دامنه سیگنال صوتی بر حسب زمان

```
sound(x, fs);
audiowrite('x.wav', x, fs);
```

ذخیره سیگنال به صورت فایل صوتی x.wav

خروجی این اجرا به شکل زیر خواهد بود:



تمرین ۳)

در این تمرین تابع p4_3 باید طوری طراحی شود که سیگنال صوتی ذخیره شده در بخش قبل را ورودی گرفته و نرخ بازپخش آن را کنترل کند. برای این کار باید نرخ نمونه برداری را در زمان پخش تغییر دهیم یعنی کافیست فرکانس نمونه برداری(fs) را در متغیر speed ضرب کنیم که در نتیجه آن نرخ خواندن نمونه های گسسته، تغییر کرده و زمان کل بازپخش سیگنال با ضریب $\frac{1}{speed}$، scale می شود.

```
function p4_3(x, speed)
  valid_speeds = 0.5 : 0.1 : 2.0;
```

بردار سرعت‌های مجاز

```
  if ismember(speed, valid_speeds) == false
     error('it is not a valid speed!');
  end
```

بررسی این که ورودی حتما در بازه مجاز باشد

```
     [audio_data, fs] = audioread(x);
```

استخراج داده‌های صوتی و فرکانس نمونه‌برداری از فایل

```
  sound(audio_data, fs * speed);
end
```

پخش صدا با تغییر نرخ نمونه‌برداری

نتیجه اجرای تابع به ازای چند سرعت متفاوت:

سرعت 0.5 برابر:

در این حالت سرعت پخش صدا نصف می‌شود و در نتیجه مدت زمان پخش دو برابر خواهد شد. از طرفی، فرکانس‌های صدا نیز نصف می‌شوند و صدا به شکل بم‌تر و کشیده‌تر شنیده می‌شود.

سرعت 1.5 و 2 برابر:

در این حالت صدا با سرعت بیشتری پخش می‌شود و مدت زمان آن کاهش پیدا می‌کند. برای مثال، در سرعت دو برابر، مدت زمان پخش به نصف می‌رسد. همچنین فرکانس‌های صدا افزایش پیدا می‌کنند و صدا زیرتر و نازک‌تر شنیده می‌شود.